

STBD-56 — Validação Arquitetura Banco PostgreSQL do Novo Monitor

Documento revisado: STBD-56

Escopo da revisão: modelo de dados, correção do mecanismo de sincronismo incremental, desempenho, operação, segurança e governança.

Data: 2026-08-20

Revisor: Tiago J. Adami (adami@powerdba.com.br)

Arquitetura identificada: O hub Postgres é **um por cliente** (cada instalação CISS tem o seu). Tratamento de exclusões, granularidade do watermark no PDV, filtro por empresa na API, modelo de concorrência do Sync e volumetria permanecem **indefinidos** e estão registrados como pontos em aberto na seção 8.

Edição e revisão

O texto original foi revisado e formatado pelo SabeTudo - Modelo de Inteligência da PowerDBA especialista em bancos de dados.

powerdba-db-pgsql:21b

1. Resumo executivo

A proposta é funcional no caso simples (um único processo de carga, transações curtas, sem exclusões) e o uso de `INSERT ... ON CONFLICT ... DO UPDATE ... WHERE` para evitar versionamento redundante está correto e é a melhor parte do desenho. Entretanto, **o modelo, como descrito, não garante entrega completa dos registros ao PDV.**

Existe uma falha de correção estrutural — não é um erro de implementação, é uma característica de sequências no PostgreSQL — que provoca **perda silenciosa e permanente de registros** sempre que houver duas transações de escrita simultâneas.

Silenciosa é a palavra crítica: não gera erro, não gera log, e a divergência só aparece no caixa, em forma de preço ou promoção errada.

Além disso, há 4 pontos que precisam de atenção antes de qualquer implementação:

1. Exclusões (`DELETE`) não são capturadas - é intencional?
2. Como será a carga inicial?
3. Notei que não há índice em `IDNSU` , isso precisa ser criado impreterivelmente.
4. Em um eventual restore de backup do banco reinicia as sequences abaixo do NSU já armazenado nos PDVs, o que congela a sincronização para sempre.

Um agravante de contexto: **o hub Postgres é um por cliente**. Toda a estrutura — 129 objetos por instalação — será replicada em toda a base instalada, sem DBA dedicado acompanhando cada ambiente. Isso muda a natureza das falhas silenciosas descritas acima: elas não ocorrem em um lugar onde alguém eventualmente percebe, ocorrem em N lugares onde ninguém está olhando, e chegam à CISS como chamado de "preço errado no caixa" sem causa aparente. Ver seção 6.5.

Seis definições centrais do desenho também permanecem em aberto — modelo de concorrência da carga, tratamento de exclusões, granularidade do watermark no PDV, filtro por empresa na API, volumetria e largura do inteiro no PDV. Estão consolidadas na **seção 8** como perguntas objetivas, e três delas determinam qual das correções propostas é a adequada.

Recomendação: **não aprovar para produção no formato atual**. As correções são pontuais e de baixo custo (seções 2.1, 2.2, 2.4, 2.6 e 4.1) e preservam o desenho geral — não é necessário abandonar a abordagem de NSU. A revisão só pode ser fechada depois das respostas da seção 8.

1.1 Matriz de riscos

#	Item	Severidade	Efeito
2.1	Ordem de commit ≠ ordem de <code>nextval</code>	CRÍTICO	Registro nunca entregue ao PDV
2.2	<code>DELETE</code> não capturado — <i>em</i>	CRÍTICO	PDV vende com dado excluído na

	<i>aberto</i>		origem
2.3	<code>DEFAULT 0</code> + trigger desabilitada	ALTO	Registro invisível para sempre
2.4	Restore/recriação o reinicia sequence	ALTO	Sincronismo congela silenciosamente
2.5	Alteração de PK na origem	MÉDIO	Registro órfão permanente
2.6	Carga inicial / backfill inexistente	ALTO	Registros legados nunca trafegam
3.1	Sem <code>ORDER BY</code> / <code>LIMIT</code> / paginação	ALTO	Estouro de memória na API e no PDV
3.2	Watermark sem ponto consistente — <i>em aberto</i>	MÉDIO	PDV com pai sem filho (combo sem item)
4.1	Ausência de índice em <code>IDNSU</code>	ALTO	<i>Seq scan</i> em cada poll de cada PDV
4.2	<code>IDNSU</code> indexado bloqueia HOT update	MÉDIO	Amplificação de escrita e bloat de índice
4.5	Falso positivo de alteração (CHAR/NUMERIC)	ALTO	Churn total da tabela a cada ciclo
	Aplicação conectando		Ausência de

5.1	como postgres	ALTO	menor privilégio
6.1	3 objetos x 43 tabelas = 129 objetos	MÉDIO	Divergência de código (já ocorrendo)
6.5	Hub por cliente, sem DBA por ambiente	ALTO	Falha silenciosa replicada na base instalada
8 (Q5)	Volumetria inexistente	ALTO	Impossível dimensionar ou aprovar com critério

Itens marcados como *em aberto* dependem de definição do time CISS (seção 8) e sua severidade pode subir ou cair conforme a resposta.

2. Riscos de correção — podem causar perda silenciosa de dados

2.1 [CRÍTICO] A ordem de commit não é a ordem do `nextval` : registros são perdidos permanentemente

Este é o ponto central da revisão.

O `nextval` é executado no momento do `INSERT` / `UPDATE` (dentro do trigger), mas o registro só se torna **visível** para a API no momento do `COMMIT`. Como as transações não commitam necessariamente na mesma ordem em que consumiram os números da sequence, a consulta `WHERE IDNSU > :ultimo_nsu` pode saltar um registro definitivamente.

Cenário mínimo, com duas transações concorrentes:

```

t0  T1: BEGIN; UPDATE produto ... ;      -- trigger atribui IDNSU =
100
t1  T2: BEGIN; UPDATE produto ... ;      -- trigger atribui IDNSU =
101
t2  T2: COMMIT;                          -- registro 101 fica
visível
t3  API: SELECT ... WHERE IDNSU > 99      -- retorna somente o
registro 101
      PDV armazena ultimo_nsu = 101
t4  T1: COMMIT;                          -- registro 100 fica
visível AGORA
t5  API: SELECT ... WHERE IDNSU > 101     -- o registro 100 NUNCA
será retornado

```

O registro 100 fica permanentemente invisível para aquele PDV, até que aquele mesmo registro sofra uma nova alteração por coincidência. Não há erro, não há exceção, não há log. Em uma tabela como `PRODUTO_PRECO_MASTER` isso significa um preço desatualizado no caixa por tempo indeterminado.

Agravantes no desenho atual:

- Quanto maior a duração da transação de carga, maior a janela de exposição. Uma carga em lote longa (`descarga` do DB2) que consome milhares de NSUs e commita ao final expõe todo o intervalo.
- Basta **uma** operação manual concorrente (correção de dado, script de suporte, job de manutenção) para produzir o salto, mesmo que o Sync seja monothread.
- O problema também ocorre entre o Sync e qualquer outro processo que escreva na mesma tabela.

Isso não se resolve com ajuste de sequence (`CACHE` , `INCREMENT` , `NO CYCLE`). É consequência do fato de a sequence ser não transacional. Existem duas correções viáveis:

Opção A — Serializar a atribuição de NSU (correção mais barata). Se o Sync já é efetivamente um escritor único, basta garantir isso explicitamente. Toda transação que atribua NSU adquire um lock exclusivo no início:

```
BEGIN;
SELECT pg_advisory_xact_lock(hashtext('nsu_sync'));
-- carga do lote
COMMIT; -- lock liberado no commit
```

Com isso, duas transações que atribuem NSU nunca se sobrepõem no tempo, e ordem de `nextval` = ordem de commit. Custo: serialização total dos escritores. Se o Sync já é sequencial, o custo é zero e o lock passa a ser uma **garantia** em vez de uma suposição. Limitação: não protege de escritas feitas por processos que não adquiram o lock (exige disciplina ou revogação de `INSERT` / `UPDATE` para os demais papéis — ver 5.1).

Opção B — Janela segura via registro de lotes (permite paralelismo). Manter o `nextval` por linha, mas a API nunca serve NSU acima de um teto seguro derivado dos lotes ainda não publicados. Detalhado no **Anexo C**. É a opção recomendada se houver ou vier a haver paralelismo na carga.

Ponto de atenção para o teste de aceite: o plano de testes atual (estação local + banco na Oracle) não reproduz este defeito, porque ele só aparece sob concorrência. Sugerimos um teste dirigido: manter uma transação aberta com um `UPDATE` não commitado enquanto uma segunda transação altera e commita outro registro, e então validar se o consumidor recebe os dois registros.

2.2 [CRÍTICO] Exclusões não são capturadas

Status: em aberto. Confirmado na revisão que o tratamento de exclusões ainda não foi definido. Fica registrado como ponto de atenção formal: hoje nenhuma das 43 tabelas do escopo está coberta, e a definição precisa vir antes da implantação, não depois.

o trigger é `BIU` (`BEFORE INSERT OR UPDATE`). Um `DELETE` não altera `IDNSU` de nada — a linha simplesmente desaparece. Consequência: o PDV **nunca fica sabendo que um registro foi excluído** e continua operando com ele indefinidamente.

Impacto por tipo de tabela:

- `PROMOCAO_COMBO` , `PROMOCAO_COMBO_PRODUTO` ,
`CAMPANHA_PROMOCIONAL_*` : promoção excluída continua sendo concedida no

caixa (prejuízo financeiro direto).

- `PRODUTO_EMPRESA` : produto desvinculado da loja continua sendo vendido.
- `PRODUTO_KIT` , `PRODUTO_COMPLEMENTAR` , `CLIENTE_AUTORIZADO` : vínculo removido permanece ativo.

Também é preciso esclarecer **como o Sync detecta exclusões no DB2**, porque a resposta muda a solução. Se a estratégia vier a ser `DELETE` + recarga (ou `DELETE` do conjunto por chave-pai seguido de `INSERT`), o resultado é pior do que não tratar: as linhas recriadas recebem NSU novo, o que força todos os PDVs a rebaixar a tabela inteira a cada ciclo (ver 4.3/4.5) e, ainda assim, eles não recebem a informação da exclusão real. Se o DB2 não expõe as exclusões de forma confiável, a captura precisa ser feita no próprio Postgres, por trigger `AFTER DELETE`, e o Sync precisa de uma estratégia de detecção (comparação de chaves por faixa, ou tabela de auditoria na origem).

Correções possíveis, em ordem de preferência:

1. **Exclusão lógica com NSU** — coluna `FLEXCLUIDO` boolean NOT NULL `DEFAULT false` (ou `DHEXCLUSAO` `timestampz`), o Sync faz `UPDATE` marcando a exclusão (o trigger incrementa o NSU naturalmente), a API entrega a linha marcada, o PDV remove localmente. Requer política de expurgo (só é seguro apagar fisicamente depois que todos os PDVs passaram daquele NSU).
2. **Tabela de controle** — trigger `AFTER DELETE` grava `(tabela, chave, nsu)` em uma tabela de log única, consultada pela API. Também é o caminho para um cursor único (ver 3.2).

2.3 [ALTO] `DEFAULT 0` combinado com trigger desabilitada gera registro invisível

`IDNSU BIGINT NOT NULL DEFAULT 0` cria uma condição perigosa: qualquer linha inserida **sem** o trigger ativa fica com `IDNSU = 0` e, portanto, invisível para todo cliente cujo watermark seja maior que zero. Isso não é hipotético; as situações em que o trigger não dispara são comuns:

- `ALTER TABLE ... DISABLE TRIGGER ALL` — prática usual para acelerar carga em massa.
- `session_replication_role = 'replica'` — usado por `pg_restore -disable-triggers`, por *workers* de replicação lógica e por várias ferramentas

de ETL. **Triggers normais não disparam neste modo.**

- Tabela nova incluída no fluxo antes da criação do trigger (situação já declarada no request: *"Para algumas tabelas das relacionadas acima, não tem a estrutura atualizada ainda"*).

Possível correção (necessita testes):

```
-- garante disparo mesmo sob session_replication_role = 'replica'  
ALTER TABLE public.produto_kit ENABLE ALWAYS TRIGGER  
tg_biu_nsu_produto_kit;
```

E um monitoramento permanente de `IDNSU = 0` residual por tabela (Anexo A), tratado como alerta e não como informação. Se possível, remover o `DEFAULT 0` após o backfill (2.6), deixando a coluna sem default para que uma inserção sem trigger falhe em vez de passar silenciosamente.

2.4 [ALTO] Restore, recriação ou reconstrução do hub reinicia as sequences e congela a sincronização

Se o banco Postgres for recriado (nova instalação, *rebuild*, recarga total a partir do DB2) as sequences voltam a valores baixos. O PDV, que guarda `ultimo_nsu = 8.500.000`, passará a receber **zero registros para sempre**, porque todos os NSUs novos são menores que o watermark armazenado. Novamente: falha silenciosa, sem erro.

Correção: introduzir um identificador de instância/época (`epoch`) que a API devolve em toda resposta e que o PDV persiste junto do watermark. Se o `epoch` recebido for diferente do armazenado, o PDV zera o watermark e faz carga completa. O `epoch` pode ser `system_identifier` do cluster, ou um UUID gravado em uma tabela de controle na criação do hub — desde que nunca seja reaproveitado. Esse watermark precisa ser alterado cada vez que o banco for criado, recriado ou restaurado de um backup.

2.5 [MÉDIO] Alteração de chave primária na origem deixa registro órfão

O padrão `ON CONFLICT` trata alteração de atributos, mas não alteração de chave. Se a PK de um registro muda no DB2, o Sync insere uma linha nova e a antiga

permanece no Postgres para sempre — e, pelo item 2.2, o PDV também nunca a remove. Precisa ser confirmado se isso é possível no modelo do DB2; se for, exige tratamento explícito. Tenho uma vaga lembrança que produtos pesáveis podem sofrer alterações de IDSUBPRODUTO para caber em 4 ou 6 dígitos para as balanças.

2.6 [ALTO] Não existe plano de carga inicial (backfill) — e o filtro `> 0` exclui exatamente os registros legados

Combinação problemática: a coluna é criada com `DEFAULT 0` e a consulta da API é `WHERE IDNSU > :ID_NSU`. Um PDV novo inicia com `ID_NSU = 0`, logo `IDNSU > 0` **exclui todos os registros pré-existent**s, que estão justamente com `IDNSU = 0`. Sem um backfill ou um endpoint de carga total separado, o PDV nunca recebe a base histórica.

Alternativas:

- Backfill explícito (`UPDATE tabela SET idnsu = nextval(...)`), executado em blocos com commit intermediário. Atenção: reescreve 100% das linhas — dobra o tamanho da tabela até o autovacuum e gera volume proporcional de WAL. Precisa de janela e de planejamento por tabela (ver 4.3).
- Alterar o contrato para watermark inicial `-1` com `IDNSU >= 0`, aceitando que todas as linhas legadas compartilhem o NSU 0 (nesse caso a paginação precisa de desempate por PK — ver 3.1).
- Endpoint dedicado de carga inicial (*snapshot* + NSU corrente no momento do snapshot), com o incremental assumindo dali em diante. É o mais robusto, e resolve também o cenário de troca/formatação de PDV.

Observação positiva: `ALTER TABLE ... ADD COLUMN bigint NOT NULL DEFAULT 0` **não** reescreve a tabela no PostgreSQL 17 (otimização de default não volátil, disponível desde a 11). O DDL em si é rápido; o custo está no backfill. Ainda assim, o `ALTER` exige `ACCESS EXCLUSIVE` momentâneo — use `lock_timeout` para não enfileirar a aplicação atrás de uma transação longa.

3. Contrato da API

3.1 [ALTO] Consulta sem `ORDER BY`, sem `LIMIT` e sem

paginação

A consulta apresentada não tem limite nem ordenação:

```
SELECT IDNSU, IDSUBPRODUTO, POSICAOPRODUTO, DTALTERACAO
FROM public.PRODUTO_TABELA_RAPIDA
WHERE IDNSU > :${ID_NSU}
```

Problemas:

- A primeira chamada de um PDV novo (ou após backfill/recarga) retorna a tabela inteira em uma única resposta — estouro de memória na API e no PDV, timeout, e retrabalho total em caso de falha no meio da transferência.
- Sem `ORDER BY IDNSU` não há garantia de ordem de retorno; o PDV não pode simplesmente adotar o último NSU recebido.
- O PDV deve avançar o watermark para o `max(IDNSU)` **efetivamente recebido e aplicado**, nunca para um `max(IDNSU)` da tabela consultado no servidor — caso contrário volta a pular registros.

Correção: `ORDER BY IDNSU` (com desempate por PK se o NSU não for único) + `LIMIT` com teto configurável, retornando na resposta o `proximo_nsu` e um indicador `tem_mais`. A repetição de um registro entre páginas é aceitável e esperada, desde que o PDV aplique tudo de forma idempotente (upsert).

3.2 [MÉDIO] Watermark por tabela: não existe ponto de consistência entre tabelas relacionadas

Status: em aberto. Não está definido se o PDV guardará um NSU por tabela ou um NSU único. O request implica o primeiro (uma sequence por tabela), mas isso não está declarado — e é uma decisão de arquitetura, não de implementação.

Com uma sequence por tabela, o PDV mantém 43 cursores independentes e não há nenhum instante em que sua base local esteja consistente. É perfeitamente possível o PDV receber `PROMOCAO_COMBO` sem os respectivos `PROMOCAO_COMBO_PRODUTO`, ou `PRODUTO` sem `PRODUTO_EMPRESA`, e aplicar uma promoção parcial.

Isso pode ser aceitável se o PDV tolerar estado transitório e a aplicação for tolerante a FK ausente — mas precisa ser uma **decisão consciente e documentada**, não um

efeito colateral. Alternativas: sequence global compartilhada (cursor único, ordem global), agrupamento por "lote publicado" (Anexo C, que dá atomicidade por lote), ou tabela de log única (outbox).

3.3 [MÉDIO] `: ${ID_NSU}` sugere interpolação de string

A notação mistura *bind parameter* (`:`) com interpolação (`${ }`). Se o valor for concatenado na query, há injeção de SQL em um endpoint exposto a centenas de PDVs. Deve ser *bind parameter* tipado como `bigint`, com validação de faixa na borda da API.

3.4 [MÉDIO] Tipo do NSU no PDV

`BIGINT` no servidor só ajuda se o cliente também armazenar 64 bits. Se o PDV for legado (Delphi/C++ com `Integer` de 32 bits, ou JSON desserializado para inteiro de 32 bits), há estouro em 2.147.483.647. Considerando que a sequence avança **também nos upserts descartados** (ver 4.6), esse teto é alcançável mais rápido do que parece. Precisa ser verificado ponta a ponta, incluindo a serialização JSON (JavaScript perde precisão acima de 2^{53} — se houver qualquer camada JS no caminho, o NSU deve trafegar como string).

3.5 [MÉDIO] Custo do polling de 43 tabelas por PDV

Não existe volumetria disponível (seção 8), então o número abaixo é ilustrativo — mas a ordem de grandeza importa: 300 PDVs × 43 tabelas a cada 60 s ≈ **215 consultas/segundo** por instalação, quase todas retornando zero linhas. É carga pura de verificação, paga em todo cliente da base instalada.

Sugestão: um endpoint único de "o que mudou", que devolve o NSU máximo por tabela, de modo que o PDV só consulte as tabelas efetivamente alteradas. Isso pode vir de uma tabela de controle mantida pelo Sync ao final de cada lote (evite atualizar uma linha única a partir de muitas transações concorrentes — cria linha quente), ou, como simples *hint*, de `pg_sequences.last_value` — que é barato, mas **não serve como watermark** porque reflete valores ainda não commitados.

4. Desempenho e operação

4.1 [ALTO] Nenhum índice em IDNSU foi previsto

O request não cria índice. `WHERE IDNSU > :nsu` sem índice é *seq scan* — em `PRODUTO` e `PRODUTO_EMPRESA` isso é inviável na frequência de polling da seção 3.5.

```
CREATE INDEX CONCURRENTLY ix_produto_kit_idnsu ON
public.produto_kit (idnsu);
```

Notas:

- Use `CONCURRENTLY` para criar em tabelas já em uso.
- **BRIN não serve** aqui, apesar de o `IDNSU` ser crescente: as linhas atualizadas são regravadas em qualquer página com espaço livre, então não há correlação física entre valor e posição. Índice B-tree.
- **Em aberto:** não está definido se a API filtra por empresa/loja ou se o PDV baixa a tabela inteira e filtra localmente. Isso precisa constar do documento, porque muda o índice: com filtro, ele tem de ser composto — `(idempresa, idnsu)` — senão o *index scan* percorre e descarta as linhas das demais empresas, e o custo cresce com o número de lojas do cliente. Muda também a granularidade do watermark e o volume trafegado por PDV.

4.2 [MÉDIO] IDNSU indexado impede HOT update e amplifica a escrita

Consequência direta e pouco óbvia: no PostgreSQL, se **qualquer** coluna indexada muda, o *update* não pode ser HOT e é preciso inserir nova entrada em **todos** os índices da tabela. Como o `IDNSU` muda em toda alteração e está indexado, todo `UPDATE` do Sync passa a gravar em todos os índices daquela tabela — em tabelas largas e com muitos índices (`PRODUTO`), a amplificação de escrita e de WAL é significativa.

Além disso, o índice de `IDNSU` é monotonicamente crescente com valores antigos sendo invalidados à esquerda: as páginas iniciais esvaziam e só são recuperadas por `VACUUM`, gerando bloat contínuo de índice.

Mitigações: `fillfactor` reduzido nas tabelas de maior churn, autovacuum agressivo por tabela, `REINDEX CONCURRENTLY` periódico — ou tirar o feed de

mudanças da tabela base (padrão outbox), o que preserva o HOT update das tabelas de negócio. Este item é o argumento técnico mais forte a favor da tabela de log única.

4.3 [MÉDIO] Bloat e autovacuum não foram endereçados

Cada `UPDATE` cria uma nova versão da linha. Um ciclo que altere toda a tabela dobra seu tamanho até o autovacuum recuperar espaço. Com 43 tabelas e a possibilidade de churn total (ver 4.5), bloat tende a ser o principal problema operacional do hub. É necessário: parametrização de autovacuum por tabela

(`autovacuum_vacuum_scale_factor` na casa de 0,05 para as de maior churn), `autovacuum_vacuum_cost_limit` adequado, monitoramento de bloat, e um plano de `pg_repack` para as tabelas que já vierem infladas do backfill.

4.4 [BAIXO] Custo do trigger na carga em massa

Um trigger de linha em PL/pgSQL adiciona custo por registro em cada

`INSERT / UPDATE`. Em cargas de milhões de linhas isso é mensurável. A alternativa (atribuir `nextval` diretamente no comando do Sync) **não é recomendada aqui**, por um motivo importante: se o `IDNSU` entrar na lista de colunas do `INSERT`, ele passa a integrar a comparação do `ON CONFLICT ... WHERE (PTR.IDNSU IS DISTINCT FROM EXCLUDED.IDNSU` é sempre verdadeiro) e destrói exatamente o benefício de evitar updates redundantes. O trigger é a escolha correta; vale apenas dimensionar o custo em teste de carga.

4.5 [ALTO] Falsos positivos de alteração podem provocar churn total a cada ciclo

O `IS DISTINCT FROM` só entrega o benefício prometido se os valores comparados forem realmente equivalentes entre DB2 e Postgres. Fontes conhecidas de falso positivo em migração DB2 → PostgreSQL:

- **CHAR no DB2 é preenchido com espaços à direita.** `CHAR(10)` contendo `'ABC'` chega como `'ABC '`; comparado com um `varchar` que guarda `'ABC'`, resulta em "diferente" **sempre**. Toda linha é atualizada em todo ciclo, o NSU de tudo é incrementado, e todos os PDVs rebaixam a tabela inteira a cada ciclo — anulando o desenho e multiplicando o bloat.
- **DECIMAL / NUMERIC com escalas divergentes** — `10.5` vs `10.50` são iguais em `numeric`, mas não se houver conversão intermediária para texto ou

`double precision` .

- **TIMESTAMP** com precisão diferente, e `timestamp` vs `timestampz` com conversão de fuso.
- **String vazia vs NULL** — o DB2 não os distingue da mesma forma que o PostgreSQL em todos os caminhos de conversão.
- **DTALTERACAO** : se essa coluna for atualizada na origem a cada *toque* no registro (mesmo sem mudança de dado de negócio), ela sozinha invalida toda a proteção. Vale questionar se `DTALTERACAO` é dado de negócio necessário ao PDV ou apenas auditoria — se for auditoria, deve **sair** da cláusula de comparação (continuando a ser gravada, mas sem disparar versionamento).

Recomendação: escrever um teste que rode duas cargas idênticas em sequência e valide que a segunda produz **zero** incremento de NSU, por tabela. Esse teste é a melhor proteção contra toda esta classe de problema, e deve ser permanente (não só na homologação).

4.6 [BAIXO] Comportamentos do `ON CONFLICT` que precisam estar claros para o time

Três detalhes do padrão adotado, todos sem impacto de correção, mas que geram surpresa:

1. o trigger `BEFORE INSERT` dispara antes da detecção do conflito. Ou seja, todo upsert consome um `nextval` mesmo quando o `INSERT` se converte em `UPDATE` — e outro se o `UPDATE` acontecer. A sequence avança muito mais rápido que o número de alterações reais e os NSUs terão grandes lacunas. Lacunas são inofensivas para a semântica `>` , mas o `IDNSU` **não** é um contador de alterações e não deve ser interpretado como tal.
2. **A tupla conflitante é travada** na resolução do conflito, ainda que o `WHERE` descarte o `UPDATE` . Escritores concorrentes na mesma chave serializam — relevante se houver paralelismo por partição de chave.
3. **0 linhas afetadas é o resultado normal** quando o `WHERE` descarta o update. O Sync não pode tratar isso como erro nem como "registro inexistente".

4.7 [BAIXO] Parametrização das sequences não está explícita

Deixe explícito no script, para não depender de default: `AS bigint INCREMENT BY 1 START WITH 1 NO CYCLE CACHE 1` . Em especial: **CYCLE** **jamais** (reiniciar a

sequence recria o problema de 2.4), e `CACHE 1` — cache maior que 1 faz sessões distintas entregarem valores fora de ordem, agravando 2.1.

4.8 [BAIXO] Pool de conexões e timeouts

`database.max-pool-size=20` é por instância da API; com N instâncias mais o Sync, é preciso dimensionar `max_connections` e avaliar PgBouncer. Não há menção a `statement_timeout` nem a `idle_in_transaction_session_timeout` — o segundo é especialmente relevante aqui, porque uma transação aberta e esquecida é exatamente o gatilho de 2.1 e bloqueia o VACUUM. O `leak-detection-threshold=50000` trata sintoma, não causa.

////////////////////////////////////

5. Segurança

5.1 [ALTO] Aplicação conectando como `postgres`

`database.user=postgres` é superusuário. Além da violação de menor privilégio, isso impede um controle que resolveria parte de 2.1: com papéis separados, é possível **garantir** quem escreve.

Modelo mínimo: papel `sync_writer` com `SELECT, INSERT, UPDATE` nas tabelas do fluxo; papel `api_reader` com `SELECT` apenas. A API não deve ter permissão de escrita em nenhuma tabela do fluxo — isso elimina por construção a possibilidade de a API interferir no NSU.

5.3 [MÉDIO] TLS não declarado na conexão da aplicação

A configuração JDBC não traz `sslmode`. O default do driver PostgreSQL não exige TLS, então a aplicação pode estar trafegando credenciais e dados em claro — inclusive contra um host externo (`*.dataciss.com.br`). O request menciona que o pgAdmin4 precisa de certificado, o que indica que há TLS disponível; então a aplicação deveria usar `sslmode=verify-full` com o CA correspondente. `require` protege o transporte mas não valida a identidade do servidor (não protege contra MITM).

5.4 [MÉDIO] Exposição de rede

Host com nome público resolvível na porta 5432. Mesmo sendo ambiente de desenvolvimento, o padrão precisa ser definido agora: restrição por *security list*/firewall às origens conhecidas, ou acesso via bastion/private link. Vale confirmar também se este ambiente de desenvolvimento contém dados reais de clientes — se sim, há implicação de LGPD nas tabelas `CLIENTE`, `CLIENTE_FORMA_PAGTO`, `CLIENTE_AUTORIZADO` e `USUARIO`, e o dado deveria estar mascarado/anonimizado.

5.5 [BAIXO] Hardening da função de trigger

O corpo da função não consta do request (ver seção 8), mas dois pontos devem ser exigidos: a função deve **qualificar o schema** da sequence (ou fixar `SET search_path = pg_catalog, public`), evitando que uma sequence de mesmo nome em outro schema do `search_path` seja usada; e não deve ser `SECURITY DEFINER` sem necessidade — se for, o `search_path` fixo passa a ser obrigatório.

////////////////////////////////////

6. Manutenibilidade e governança

6.1 [MÉDIO] 129 objetos onde bastariam 44

O desenho cria, por tabela, 1 sequence + 1 função + 1 trigger: 43 tabelas × 3 = 129 objetos, com 43 funções contendo a **mesma lógica duplicada**. Esse é o tipo de duplicação que garante divergência ao longo do tempo — e o próprio request já registra o sintoma: *"Para algumas tabelas das relacionadas acima, não tem a estrutura atualizada ainda."*

Uma única função genérica recebendo o nome da sequence como argumento do trigger reduz para 43 sequences + 43 triggers + 1 função, e qualquer correção passa a ser feita em um único lugar. Ver **Anexo B**.

6.2 [MÉDIO] Divergência já existente e ausência de verificação automatizada

Não há como afirmar hoje que as 43 tabelas estão conformes. É necessário um script

de verificação por catálogo — coluna presente, tipo correto, `NOT NULL`, sequence existente, trigger existente e **habilitada**, índice presente, PK presente — executado no *deploy* e periodicamente em produção, com saída tratada como falha. Ver **Anexo A**.

6.3 [MÉDIO] Ausência de plano de versionamento, implantação e rollback

O request apresenta o DDL, mas não: scripts idempotentes, ordem de aplicação, plano de rollback, e mapeamento entre versão de banco e versão de aplicativo.

Como o hub é um por cliente (6.5), o mapeamento versão de banco ↔ versão de programa deixa de ser conveniência e passa a ser o único mecanismo capaz de responder "este cliente tem a estrutura necessária para esta versão do PDV?".

6.4 [BAIXO] "Versionamento de registros" é um nome enganoso

O modelo **não versiona registros**: mantém apenas a versão corrente de cada linha (upsert no local), mais um marcador crescente. Não é possível reconstruir o estado anterior nem saber *o que* mudou. Se em algum momento o requisito for auditoria/histórico (e em tabelas como `PRODUTO_PRECO_MASTER` isso costuma ser demandado), este desenho não atende e será preciso outra estrutura. Sugerimos renomear para algo como "controle de alterações incrementais (NSU)" para não criar expectativa equivocada nas áreas de negócio.

6.5 [ALTO] Hub por cliente: o desenho é uma frota, não um servidor

Isso não altera nenhum dos defeitos técnicos apontados, mas altera o peso de todos eles, e por isso merece tratamento próprio.

Consequências diretas:

- **129 objetos x número de instalações.** A implantação deixa de ser um script executado uma vez e passa a ser distribuição de release para toda a base instalada, com clientes em estágios diferentes. A frase do request — *"para algumas tabelas ainda não tem a estrutura atualizada"* — deixa de significar "algumas tabelas" e passa a significar "algumas tabelas em alguns clientes", uma

matriz que ninguém consegue acompanhar manualmente. Isso torna a função genérica única (6.1) e o script de conformidade (6.2) requisitos, não sugestões.

- **A API precisa negociar capacidade.** Com PDVs e hubs em versões diferentes, a API tem de detectar se o hub daquele cliente já possui a estrutura de NSU e degradar de forma controlada quando não possuir — em vez de falhar ou, pior, devolver conjunto vazio (que o PDV interpretaria como "nada mudou").
- **O `epoch` (2.4) passa a ser obrigatório.** Em uma frota, recriação de ambiente, restauração de backup, troca de servidor e migração de loja são eventos rotineiros. Sem o `epoch`, cada um desses eventos congela silenciosamente a sincronização daquele cliente.
- **Falhas silenciosas não têm quem as perceba.** Não há DBA por ambiente. Os itens 2.1, 2.3 e 2.4 não geram erro; produzem divergência que só aparece como reclamação de preço ou promoção incorreta no caixa, sem rastro que ligue o sintoma à causa. É o argumento mais forte a favor da reconciliação automática e dos alarmes da seção 7 — em frota, o mecanismo de detecção vale mais que o mecanismo de sincronismo.
- **Tuning tem de viajar no DDL.** `fillfactor` e parâmetros de autovacuum por tabela (4.3) precisam fazer parte do script de criação, porque ninguém vai ajustar N servidores manualmente. O mesmo vale para `statement_timeout` e `idle_in_transaction_session_timeout` (4.8), que devem ser definidos no papel da aplicação (`ALTER ROLE ... SET`) e não por servidor.
- **Perfis de volumetria em vez de um número único.** O dimensionamento precisa considerar o maior cliente da base, não a média — e o desenho tem de ser validado contra esse perfil.
- **A migração de versão maior vira gestão de frota.** O estudo de PostgreSQL 18 mencionado no request precisa contemplar N atualizações remotas, com janela, rollback e validação por cliente. É um projeto de porte diferente do que a menção no request sugere. Em termos de desempenho, há sim vantagens que podem ser discutidas, mas nenhuma com impacto suficiente para resolver os problemas apontados nesse documento.

7. Observabilidade e reconciliação (ausentes)

Todo mecanismo de sincronismo incremental precisa de um caminho de reparo, porque divergência vai acontecer. Não há nada previsto no request:

- **Reconciliação periódica** entre DB2, Postgres e PDV: contagem de linhas e checksum por faixa de chave (ex.: `md5(string_agg(...))` agrupado por faixa), com relatório de divergência.
- **Comando de ressincronismo total** por tabela e por PDV, acionável remotamente.
- **Métricas mínimas**: idade do watermark de cada PDV (quais estão atrasados), linhas retornadas por chamada, erros de aplicação no PDV, latência DB2 → Postgres → PDV, contagem de `IDNSU = 0`, bloat e último autovacuum por tabela.
- **Alarme de estagnação**: PDV cujo watermark não avança há X horas enquanto o NSU do hub avança — é exatamente o sintoma de 2.1 e de 2.4.

8. Pontos em aberto — definições pendentes por parte da CISS

Os itens abaixo não estão no request e permaneceram indefinidos na revisão. Não são pedidos de esclarecimento burocrático: cada um deles altera uma decisão técnica concreta, e três deles (Q1, Q2 e Q3) determinam **qual** das correções propostas deve ser adotada. Enquanto não houver resposta, a revisão não pode ser fechada e o item correspondente permanece com a severidade máxima estimada.

Q1 — Qual é o modelo de concorrência de escrita do Sync? Um único processo com uma transação por lote? Uma transação por registro? Workers paralelos por tabela? Workers paralelos que possam escrever na mesma tabela? *Por que importa:* define a probabilidade e a frequência da perda de registros do item 2.1, e escolhe a correção. Escritor único e serializado → basta o lock consultivo (Opção A, praticamente sem custo). Qualquer paralelismo → é preciso a janela segura por lote (Opção B, Anexo C). Também é preciso saber se **algum outro processo** escreve nessas tabelas: job de manutenção, correção manual, suporte, outro produto.

Q2 — Como as exclusões feitas no DB2 devem ser propagadas? Elas existem nessas 43 tabelas? Se existem, como o Sync as detecta hoje? *Por que importa:* é o item 2.2. Se as exclusões existem e não forem tratadas, o PDV opera indefinidamente com promoções, vínculos e produtos que já não existem na origem. A resposta também define se cabe exclusão lógica (mais simples) ou tabela de tombstones (mais completa).

Q3 — O PDV guardará um NSU por tabela ou um NSU único? *Por que importa:* é o item 3.2. Um NSU por tabela significa 43 cursores independentes e nenhum instante de consistência entre tabelas relacionadas — o PDV pode ter o combo sem os itens do combo. Um NSU único (sequence compartilhada ou lote publicado) resolve isso, simplifica o cliente e viabiliza a chamada única da seção 3.5, ao custo de acoplar as tabelas em uma ordem global.

Q4 — A API filtra por empresa/loja ou o PDV baixa a tabela inteira? *Por que importa:* item 4.1. Com filtro, o índice precisa ser composto (`idempresa, idnsu`) e o watermark passa a ser por (tabela, empresa). Sem filtro, cada PDV trafega o dado de todas as lojas do cliente. Precisa constar explicitamente do documento de projeto, tabela a tabela se o comportamento variar.

Q5 — Qual é a volumetria, por perfil de cliente? Número de PDVs por instalação, intervalo de polling, número de linhas e taxa de alteração das maiores tabelas (`PRODUTO`, `PRODUTO_EMPRESA`, `PRODUTO_PRECO_MASTER`). *Por que importa:* sem isso não é possível validar a afirmação do request de que `BIGINT` é adequado "ao volume de registros movimentados", nem dimensionar o custo do polling (3.5), nem definir o tamanho de página da API, nem parametrizar autovacuum. Em um cenário de hub por cliente (6.5), o número relevante é o do **maior** cliente da base, não a média. Recomendamos levantar isso a partir da base instalada antes de qualquer teste de desempenho — caso contrário o teste mede um cenário que não existe.

Q6 — O PDV armazena o NSU em inteiro de 64 bits ponta a ponta? Incluindo a serialização (JSON/JavaScript perde precisão acima de 2^{53}) e o tipo da coluna no banco local do PDV. *Por que importa:* item 3.4. Como a sequence avança também nos upserts descartados, o teto de 32 bits é alcançável.

Q7 — Todas as 43 tabelas possuem PK ou índice único equivalente à chave do DB2? *Por que importa:* o `ON CONFLICT` exige um *arbiter* exato. Tabelas filhas de chave composta precisam de todas as colunas no conflict target, e coluna anulável na chave não deduplica no PostgreSQL (pode ser necessário `NULLS NOT DISTINCT`). Basta uma tabela fora do padrão para duplicar registros silenciosamente. Isto pode ser verificado por catálogo (Anexo A) em vez de por levantamento manual.

Q8 — Qual é o corpo da função `UPDATE_NSU_<tabela>()` e a definição completa do trigger? O request traz apenas as assinaturas. É preciso confirmar `FOR EACH ROW`, `BEFORE` (um `AFTER` não consegue alterar `NEW`), `RETURN NEW`, qualificação de schema da sequence e ausência de lógica condicional por `TG_OP`.

Sem isso, a revisão dessa parte específica fica pendente.

9. Recomendações prioritizadas

P0 — bloqueadores para produção

1. Fechar as definições Q1 a Q4 da seção 8. São pré-requisito dos itens 2 e 3 abaixo: sem elas, a correção seria escolhida no escuro.
2. Corrigir a ordem de commit vs. `nextval` (2.1): lock de serialização (Opção A) se o escritor for único, ou janela segura por lote (Opção B / Anexo C) se houver paralelismo.
3. Definir e implementar a propagação de exclusões (2.2).
4. Criar índice B-tree em `IDNSU` em todas as tabelas do escopo (4.1).
5. Definir o caminho de carga inicial e executar o backfill, ou criar endpoint de snapshot (2.6).
6. `ENABLE ALWAYS` nas triggers e monitoramento de `IDNSU = 0` (2.3).
7. `ORDER BY` + `LIMIT` + paginação e *bind parameter* no endpoint (3.1, 3.3).
8. Separar papéis `sync_writer` / `api_reader` e exigir `sslmode=verify-full` (5.1, 5.3).

P1 — antes do rollout na base instalada

1. `epoch` de instância validado pelo PDV — obrigatório em cenário de hub por cliente (2.4, 6.5).
2. Levantar a volumetria por perfil de cliente e refazer o dimensionamento contra o maior cliente (Q5, 6.5).
3. Teste de carga idempotente ("segunda carga idêntica = zero NSU novo") como teste permanente (4.5).
4. Função de trigger genérica única + script de conformidade por catálogo executado no deploy de cada cliente (6.1, 6.2, 6.5).
5. Negociação de capacidade na API, para hubs que ainda não tenham a estrutura (6.5).
6. Submeter a alteração ao processo de versionamento de banco e definir versão mínima por release (6.3).
7. Embutir no DDL os parâmetros de autovacuum e `fillfactor` por tabela, e os timeouts no papel da aplicação; plano de `pg_repack` pós-backfill (4.3, 4.8, 6.5).
8. Reconciliação periódica automática e comando de ressincronismo total (7) — em

frota, é o que transforma falha silenciosa em alarme.

9. Validar o inteiro de 64 bits ponta a ponta até o PDV (3.4, Q6).

P2 — melhorias

1. Endpoint "o que mudou" para reduzir o polling de 43 tabelas (3.5).
2. Avaliar a tabela de log única (outbox) como evolução — resolve exclusão, cursor único, consistência entre tabelas e bloat de índice de uma só vez (2.2, 3.2, 4.2).
3. Renomear o conceito (6.4).

Sobre a migração para PostgreSQL 18: a 18 é GA desde setembro de 2025 (18.6 em agosto de 2026) e a 19 está em beta 3, com previsão de GA para setembro/outubro de 2026. Duas observações: (a) o gargalo deste projeto é de desenho, não de versão — nenhuma melhoria da 18 corrige os itens da seção 2, e o estudo de desempenho só produz conclusão útil **depois** de índice, autovacuum e paginação estarem corretos; (b) como a 19 sai em semanas, faz sentido decidir agora entre ir para a 18 e ficar nela por um ciclo, ou aguardar a maturidade da 19 (tipicamente algumas minor releases) e evitar duas migrações seguidas. Em qualquer caso, é um projeto separado, com `pg_upgrade`, validação de extensões e plano de rollback próprios — e, com hub por cliente, trata-se de atualizar uma frota inteira, não um servidor (6.5). Na Oracle, upgrade é feito com cliques, sem esforço, mas requer janela de manutenção.

Anexo A — Script de conformidade por catálogo

Verifica, para cada tabela do escopo, se toda a estrutura de NSU está presente e ativa.

Saída esperada: todas as colunas booleanas em `true` e `trigger_estado = 'A'`.

```
WITH escopo(tabela) AS (  
    VALUES  
        ('produto'), ('produto_kit'), ('produto_tabela_rapida'),  
        ('produto_empresa'), ('produto_preco_master')  
    -- completar com as 43 tabelas do escopo  
)  
  
alvo AS (  
    SELECT e.tabela,
```

```

        c.oid AS relid
    FROM escopo e
    LEFT JOIN pg_class c
        ON c.relname = e.tabela
        AND c.relnamespace = 'public'::regnamespace
        AND c.relkind = 'r'
)
SELECT
    a.tabela,
    a.relid IS NOT NULL AS tabela_existe,
    EXISTS (
        SELECT 1 FROM pg_attribute att
        WHERE att.attrelid = a.relid
            AND att.attname = 'idnsu'
            AND att.atttypid = 'bigint'::regtype
            AND att.attnotnull
            AND att.attnum > 0
            AND NOT att.attisdropped
    ) AS coluna_ok,
    EXISTS (
        SELECT 1 FROM pg_class s
        WHERE s.relkind = 'S'
            AND s.relname = 'nsu_' || a.tabela
            AND s.relnamespace = 'public'::regnamespace
    ) AS sequence_ok,
    EXISTS (
        SELECT 1
        FROM pg_index ix
        JOIN pg_attribute att
            ON att.attrelid = a.relid
            AND att.attnum = ix.indkey[0]
        WHERE ix.indrelid = a.relid
            AND att.attname = 'idnsu'
    ) AS indice_idnsu_ok,
    EXISTS (
        SELECT 1 FROM pg_constraint co
        WHERE co.conrelid = a.relid
            AND co.contype = 'p'
    ) AS pk_ok,
    (
        SELECT string_agg(tg.tgname || '=' || tg.tgenabled, ', ')
        FROM pg_trigger tg
        WHERE tg.tgrelid = a.relid
    )

```

```
        AND NOT tg.tgisinternal
    ) AS trigger_estado    -- O=origin (nao dispara em replica),
    A=always, D=disabled
    FROM alvo a
    ORDER BY 1;
```

Verificação complementar de dados, que deve retornar zero em todas as tabelas — qualquer valor maior que zero indica linhas invisíveis para clientes já sincronizados (item 2.3):

```
SELECT 'produto' AS tabela, count(*) AS linhas_sem_nsu FROM
public.produto WHERE idnsu = 0
UNION ALL
SELECT 'produto_kit', count(*) FROM public.produto_kit WHERE idnsu
= 0
-- ... demais tabelas
ORDER BY 2 DESC;
```

Anexo B — Função genérica única em lugar de 43 funções

```

CREATE OR REPLACE FUNCTION public.fn_set_nsu()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog, public
AS $$
BEGIN
    -- opcional: nao versiona update que nao altera nada de fato,
    -- protegendo contra escritores que nao usem ON CONFLICT ...
WHERE.
    -- Atencao: RETURN NULL cancela o UPDATE (0 linhas afetadas).
    IF TG_OP = 'UPDATE'
        AND to_jsonb(NEW) - 'idnsu' = to_jsonb(OLD) - 'idnsu' THEN
        RETURN NULL;
    END IF;

    NEW.idnsu := nextval(TG_ARGV[0]::regclass);
    RETURN NEW;
END;
$$;

```

Por tabela, passam a existir apenas a sequence, o trigger e o índice:

```

CREATE SEQUENCE public.nsu_produto_kit
    AS bigint INCREMENT BY 1 START WITH 1 NO CYCLE CACHE 1;

CREATE TRIGGER tg_biu_nsu_produto_kit
    BEFORE INSERT OR UPDATE ON public.produto_kit
    FOR EACH ROW
    EXECUTE FUNCTION public.fn_set_nsu('public.nsu_produto_kit');

-- garante disparo tambem sob session_replication_role = 'replica'
ALTER TABLE public.produto_kit
    ENABLE ALWAYS TRIGGER tg_biu_nsu_produto_kit;

CREATE INDEX CONCURRENTLY ix_produto_kit_idnsu
    ON public.produto_kit (idnsu);

```

Anexo C — Janela segura de NSU (correção de 2.1 preservando paralelismo)

Ideia: registrar a faixa de NSU de cada lote de carga e só publicar para a API o que estiver abaixo do menor lote ainda não confirmado. O `IDNSU` continua único por linha e a paginação por *keyset* continua simples.

```
CREATE TABLE public.nsu_lote (  
    id_lote      bigserial PRIMARY KEY,  
    tabela       text          NOT NULL,  
    nsu_inicial  bigint        NOT NULL,  
    dh_inicio    timestamptz NOT NULL DEFAULT clock_timestamp(),  
    dh_fim       timestamptz,  
    publicado    boolean       NOT NULL DEFAULT false  
);  
  
CREATE INDEX ix_nsu_lote_pendente  
    ON public.nsu_lote (tabela, nsu_inicial)  
    WHERE NOT publicado;
```

Fluxo do Sync, por lote:

1. Antes de iniciar a carga, em transação própria: registrar o lote com `publicado = false` e `nsu_inicial = coalesce(last_value, 0) + 1`, lido de `pg_sequences` para a sequence daquela tabela — e **commitar**.
2. Executar a carga do lote (o trigger atribui os NSUs) e commitar.
3. Em transação própria: `UPDATE public.nsu_lote SET publicado = true, dh_fim = clock_timestamp() WHERE id_lote = :id_lote`.

O passo 1 não precisa ser exato: `nsu_inicial` só precisa ser um **limite inferior** do primeiro NSU que o lote vai consumir. Se outro lote alocar valores nesse intervalo, o efeito é apenas tornar a janela mais conservadora, nunca insegura.

Consulta da API:

```

WITH janela AS (
    SELECT coalesce(
        (SELECT min(nsu_inicial) - 1
         FROM public.nsu_lote
         WHERE tabela = 'produto_tabela_rapida'
          AND NOT publicado),
        9223372036854775807
    ) AS nsu_seguro
)
SELECT p.idnsu, p.idsubproduto, p.posicaoproduto, p.dtalteracao
FROM public.produto_tabela_rapida p
CROSS JOIN janela j
WHERE p.idnsu > :id_nsu
      AND p.idnsu <= j.nsu_seguro
ORDER BY p.idnsu
LIMIT :limite;

```

Por que é seguro: qualquer NSU menor que o `nsu_inicial` de um lote foi necessariamente alocado antes daquele lote começar. Logo, ao servir apenas até `min(nsu_inicial dos lotes não publicados) - 1`, nenhum valor pertencente a um lote ainda não confirmado pode ser ultrapassado pelo watermark do PDV. A garantia vale inclusive com múltiplos lotes concorrentes.

Consequências a considerar: (a) a entrega ao PDV passa a ter a latência do lote mais lento em aberto — é preciso alarme para lote pendente há muito tempo; (b) lote abortado precisa ser fechado (publicado ou cancelado) por um processo de limpeza, senão trava a janela permanentemente; (c) a tabela `nsu_lote` precisa de expurgo dos lotes antigos já publicados.